



DOCTORADO EN
**INTELIGENCIA
ARTIFICIAL**



UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA



UCSC



UNIVERSIDAD DEL BÍO-BÍO

HLD & HLS

Hardware Language Description & High Level Synthesis

Doctorado en Inteligencia Artificial

Procesadores de Alto Rendimiento para IA

Profesor: Jorge Portilla G.

Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile

Fundamentos de arquitecturas computacionales

Capas de abstracción



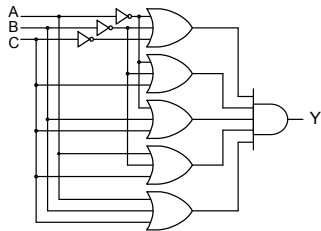
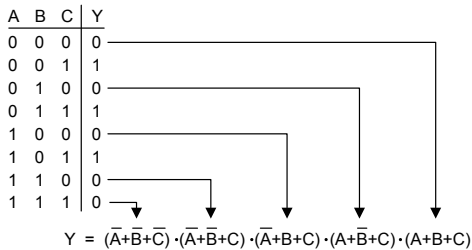
Application Software	<code>>"hello world!"</code>
Operating Systems	
Architecture	
Micro-architecture	
Logic	
Digital Circuits	
Analog Circuits	
Devices	
Physics	

► Especificación

- Tablas de verdad
- Diagramas de estado

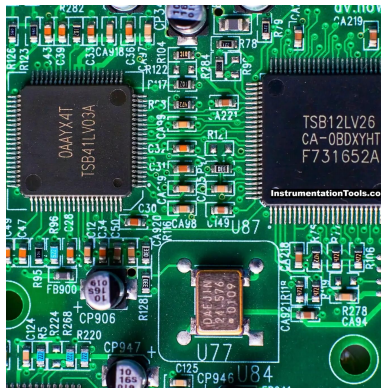
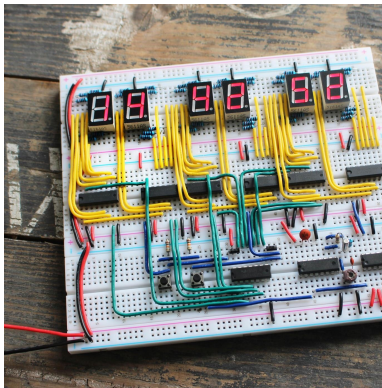
► Diseño

- Optimización local y traducción manual a puertas lógicas
- Captura esquemática

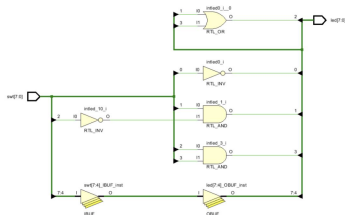


► Implementación

- Mapeo a puertas discretas
- Prototipo, pruebas y depuración (placa de pruebas)
- Implementación final



```
module logic_design (  
    input  logic [7:0] swt,  
    output logic [7:0] led  
);  
  
    // Operaciones lógicas individuales  
    assign led[0] = ~swt[0];  
    assign led[1] = swt[0] & swt[1];  
    assign led[2] = ~swt[2] | swt[1];  
    assign led[3] = swt[2] & swt[3];  
  
    // Asignación directa con buffers a los LEDs 7:4  
    assign led[7:4] = swt[7:4];  
  
endmodule
```

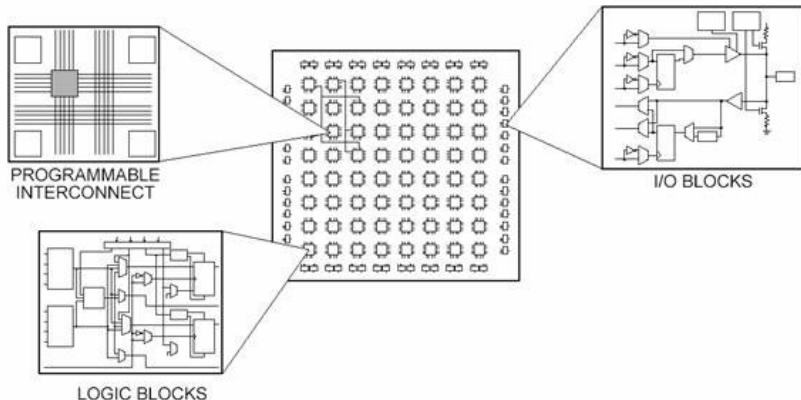


► **¡Los HDL no son lenguajes de programación!**

- Estás describiendo un sistema
- No se compila un código HDL
- No se ejecuta un código HDL

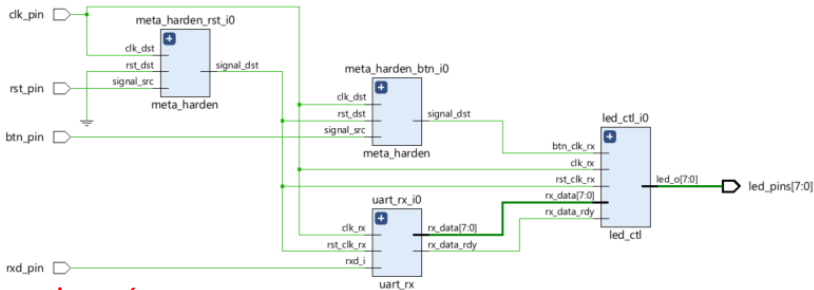
► **¡El software no obedece las leyes de la física, el hardware sí!**

Basics of Field-Programmable Gate Arrays (FPGA)



- Equivalente a millones de puertas lógicas discretas encapsuladas en un chip diminuto.
- ¿Cómo gestionar la complejidad?

Jerarquía, Modularidad y Regularidad



- **Jerarquía**

Divide un sistema complejo en módulos, y luego subdivide esos módulos hasta que las piezas sean fáciles de entender.

- **Modularidad**

Los módulos deben tener funciones e interfaces bien definidas, de forma que puedan conectarse fácilmente sin efectos inesperados.

- **Regularidad**

Promueve la uniformidad entre módulos, permitiendo que los módulos comunes se reutilicen muchas veces.

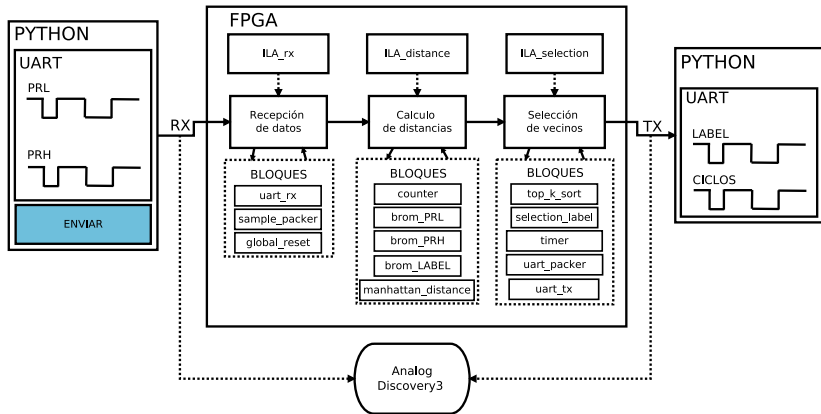
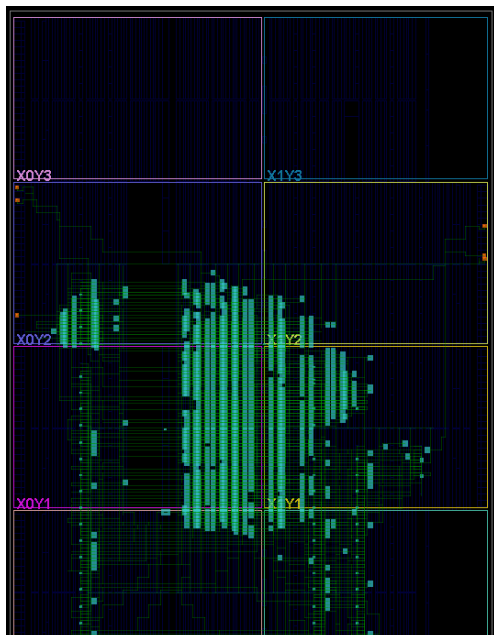


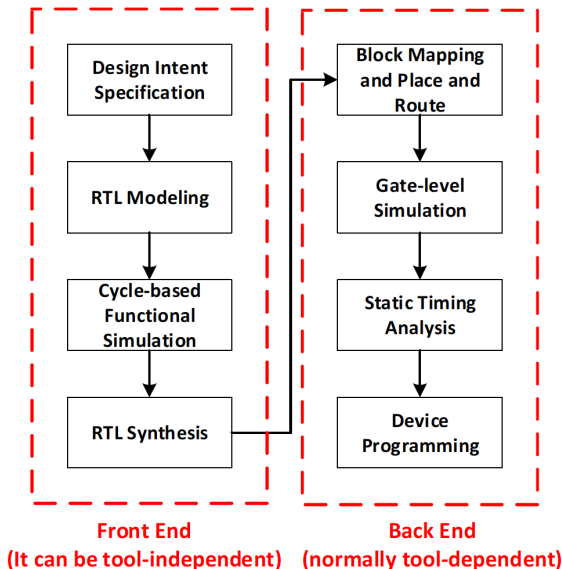
Figura 1: Arquitectura de monitoreo y validación en hardware, que integra ILAs internos y validación externa mediante un dispositivo Analog Discovery 3.



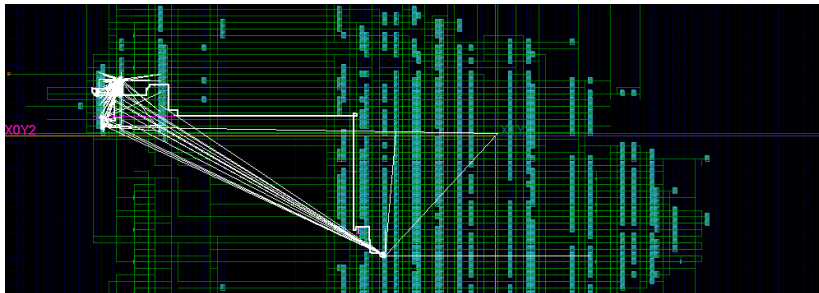
Distribución espacial de la lógica implementada sobre la FPGA, concentrada principalmente en la zona central.



Flujo de diseño típico en FPGA



Timing - Critical path



	WNS	WHS	WBSS
Valor (ns)	0,404	0,037	9,126

- ▶ WNS: Worst Negative Slack
- ▶ WHS: Worst Hold Slack
- ▶ WBSS: Worst Best Setup Slack

High-level synthesis

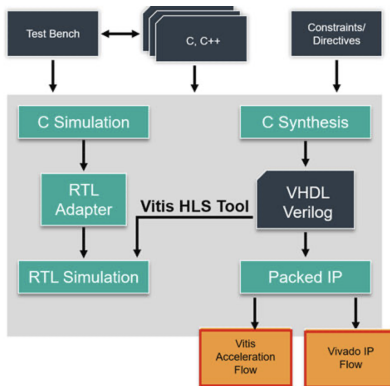
Documentación:

Vitis High-Level Synthesis User Guide [▶ Enlace](#)

- ▶ **Vitis HLS Introductory Examples** [▶ Repositorio](#) Ejemplos con guías de codificación, patrones comunes y técnicas de optimización.
- ▶ **Vitis Application Acceleration Development Flow Tutorials** [▶ Repositorio](#) Tutoriales que enseñan conceptos del flujo de herramientas y desarrollo de aplicaciones.

High-level synthesis

- Arquitectura el algoritmo basado en los principios de diseño
- (C-Simulación) Verificar el código C/C++ con el testbench en C/C++
- (C-Síntesis) Generar el RTL utilizando HLS
- (Co-Simulación) Verificar el kernel generado con las salidas en C++
- (Análisis) Revisar los reportes de síntesis HLS y de co-simulación, analizar
- Repetir los pasos anteriores hasta cumplir con las metas de desempeño



La **HLS** permite obtener una descripción en HDL a partir de un lenguaje de alto nivel (C/C++ o MatLab), reduciendo la necesidad de escribir código RTL manual.

- ▶ Reduce tiempo de diseño digital.
- ▶ Permite aplicar directivas sin alterar la funcionalidad.
- ▶ Facilita la implementación de cálculos complejos.

Limitaciones:

- ▶ Menor control fino sobre el hardware comparado con RTL.
- ▶ No es ideal para tareas con sincronía estricta (ej. protocolos).

- ▶ **Módulo:** unidad jerárquica que implementa una función o bucle.
- ▶ **Intervalo de Inicialización (II):** ciclos entre iteraciones de un bucle.
- ▶ **Planificación:** orden (potencialmente paralelo) de ejecución de módulos.
- ▶ **Trip count:** número de ejecuciones de un bucle.
- ▶ **Pragma:** directiva para guiar la herramienta en la generación de hardware.

Los **pragmas** permiten guiar la herramienta de HLS en la generación de hardware sin modificar la funcionalidad del diseño.

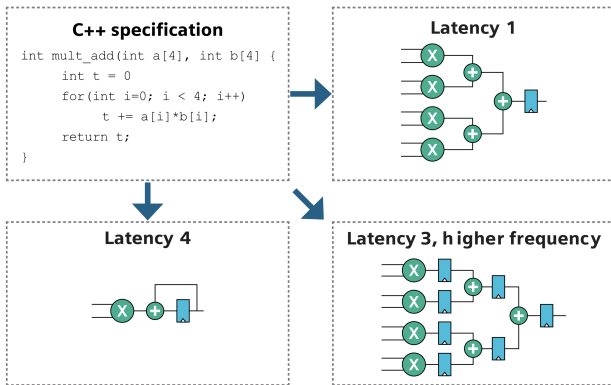
Pragmas

- ▶ **PIPELINE**: paraleliza un bucle permitiendo iniciar nuevas iteraciones antes de finalizar las anteriores.
- ▶ **UNROLL**: ejecuta múltiples iteraciones de un bucle en paralelo (factor de desenrollamiento).
- ▶ **ARRAY_PARTITION**: habilita accesos múltiples a un arreglo en el mismo ciclo.
- ▶ **LOOP_MERGE**: fusiona bucles con igual número de iteraciones en un mismo módulo.
- ▶ **LOOP_FLATTEN**: convierte bucles anidados en un solo bucle equivalente.
- ▶
- ▶ **LOOP_TRIPCOUNT**: fija cotas de iteraciones para bucles variables, útil en reportes de HLS.

Tres Paradigmas para Programar FPGAs

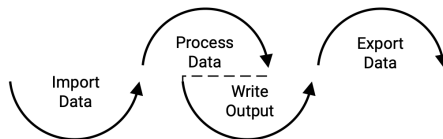
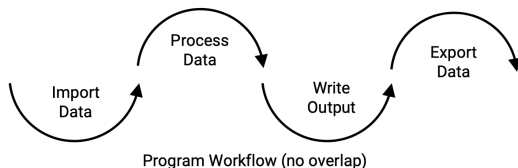
Además de los HDL, hoy en día se emplean herramientas de **High-Level Synthesis (HLS)** que permiten programar en C/C++ y traducir a Verilog/VHDL.

- Retienen ventajas del lenguaje de alto nivel.
- Reducen la curva de aprendizaje para diseñadores de software.



Paradigma 1: Producer-Consumer

- ▶ Inspirado en el paralelismo tipo `fork/join`¹ de software multithread.
- ▶ Un módulo (**productor**) genera datos que otro módulo (**consumidor**) procesa.
- ▶ Comunicación eficiente mediante **buffers FIFO o Ping-Pong**².
- ▶ Permite **solapamiento** de tareas independientes: mientras se procesa un item, otro puede escribirse en salida.

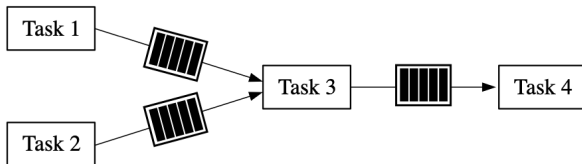


¹similar a *divide and conquer*

²doble buffer para alternar producción y consumo de bloques completos

Paradigma 2: Streaming Data

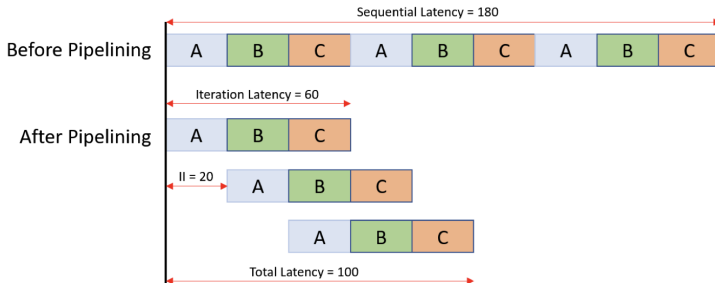
- ▶ Los datos fluyen de manera continua (*streams*).
- ▶ Comunicación unidireccional entre productor y consumidor.
- ▶ Los **FIFOs/PIPOs** permiten asincronía y paralelismo.
- ▶ Ventajas:
 - Código más limpio y secuencial.
 - Extracción automática de paralelismo.
 - Mayor **throughput** al procesar datos en flujo.



X25608-073021

Paradigma 3: Pipelining

- ▶ Similar a una línea de producción: cada etapa ejecuta una parte de la tarea.
- ▶ Aumenta el **throughput** aunque la **latencia total** no disminuya.
- ▶ Claves:
 - Balancear la duración de cada etapa.
 - **Iteration Latency**: tiempo hasta el primer resultado.
 - **Initiation Interval (II)**: tiempo entre resultados consecutivos.



- ▶ Los tres paradigmas no son excluyentes:
 - **Producer–Consumer**: tareas en paralelo comunicadas por buffers.
 - **Streaming**: comunicación continua entre módulos.
 - **Pipelining**: procesamiento escalonado dentro de funciones/bucles.
- ▶ Al combinarlos:
 - Se maximiza el paralelismo disponible.
 - Se logra alto rendimiento con bajo consumo.
 - Se obtiene un **diseño jerárquico eficiente**.

Herramienta que transforma funciones en C/C++ en RTL optimizado para lógica programable.

► Ventajas:

- Desarrollo y validación a nivel algorítmico (C-level).
- Uso de *pragmas* para controlar rendimiento.
- Exploración del espacio de diseño sin reescribir código.
- **Retargeting** hacia múltiples plataformas.

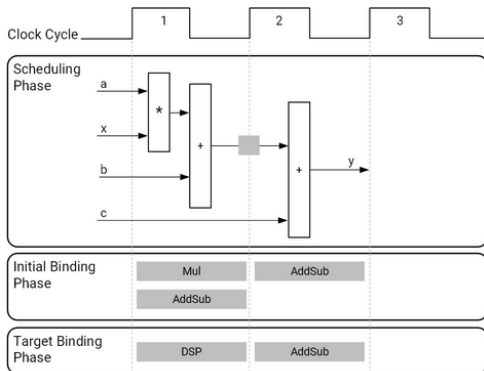
Etapas de HLS:

1. **Scheduling:** Asigna operaciones a ciclos de reloj, considerando dependencias, latencia y recursos.
2. **Binding:** Mapea operaciones a recursos físicos (ej. multiplicadores combinatoriales o pipelinizados).
3. **Control Logic Extraction:** Construcción del FSM que ordena la ejecución en el hardware.

HLS permite diseñar arquitecturas eficientes sin escribir manualmente RTL.

Scheduling & Binding

```
1  int foo(char x, char a, char b, char c) {  
2      char y;  
3      y = x*a + b + c;  
4      return y;  
5  }
```



X14220-052220

Pipelining Loops

```
1 vadd: for(int i = 0; i < len; i++) {  
2   c[i] = a[i] + b[i];  
3 }
```

```
1 vadd: for(int i = 0; i < len; i++) {  
2   #pragma HLS PIPELINE  
3   c[i] = a[i] + b[i];  
4 }
```

- ▶ Por defecto, una nueva iteración de un bucle comienza cuando termina la anterior.
- ▶ **Pipelining** permite que iteraciones se solapen: una nueva empieza antes de que acabe la anterior.
- ▶ Con `#pragma HLS PIPELINE`, el compilador inserta registros para habilitar solapamiento de operaciones.

Initiation Interval (II)

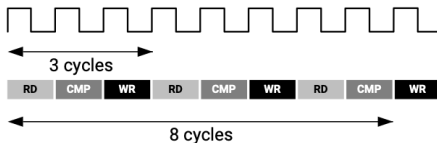
```
1 vadd: for(int i = 0; i < len; i++) {  
2   #pragma HLS PIPELINE II=1  
3     c[i] = a[i] + b[i];  
4 }
```

- ▶ **II (Initiation Interval)**: número de ciclos entre el inicio de dos iteraciones consecutivas en un bucle pipelineado.
- ▶ Ejemplo:
 - $II = 2 \Rightarrow$ nueva iteración comienza 2 ciclos después.
 - $II = 1 \Rightarrow$ nueva iteración comienza en el ciclo siguiente (ideal).
- ▶ Vitis HLS intenta alcanzar $II = 1$ por defecto.
- ▶ Se puede especificar un II objetivo en el pragma:

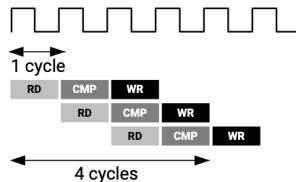
```
#pragma HLS PIPELINE II=1
```

Loop Pipelining

```
void func(m,n,o) {  
    for (i=2;i>=0;i--) {  
        op_Read;  
        op_Compute;  
        op_Write;  
    }  
}
```



(A) Without Loop Pipelining



(B) With Loop Pipelining

X14277-100620

- **Secuencial ($II=3$)**: cada iteración inicia cuando la anterior termina.
- **Pipeline ($II=1$)**: nueva iteración comienza cada ciclo... menor latencia y mayor throughput, sin más recursos.
- **IMPORTANT!** Pipelining un bucle provoca que los bucles **anidados dentro se desenrollen (unroll) automáticamente**.

- ▶ Si hay dependencias de datos, puede no lograrse $II = 1$.
- ▶ Dependencias:
 - **Intra-iteración**: dentro de la misma iteración.
 - **Inter-iteración**: entre iteraciones consecutivas (loop-carried).
- ▶ Ejemplo extremo: la condición de salida depende del **resultado previo**... la siguiente iteración no puede comenzar hasta que termine la anterior.

La siguiente iteración depende del resultado de la anterior (no pipelineable):

```
1 Minim_Loop: while (a != b) {  
2   if (a > b) a -= b;  
3   else      b -= a;  
4 }
```

- ▶ No se puede **pipelinear**: la iteración $k+1$ necesita que termine la k .

Pipelining Automático de Bucles

`config_compile -pipeline_loops` (configuración por GUI) permite pipeline automático por `#` de iteraciones.

- **Default:** 64. Bucles con ≥ 64 iteraciones se pipelinean automáticamente.
- Puedes cambiar el umbral (p.ej., 6) para pipelinear bucles internos pequeños.

Código original:

```
1  for (y = 0; y < 480; y++) {  
2      for (x = 0; x < 640; x++) {  
3          for (i = 0; i < 5; i++) {  
4              // do something 5 times ...  
5          }  
6      }  
7  }
```

Con `-pipeline_loops 6` (equivalente a agregar pragma):

```
1  for (y = 0; y < 480; y++) {  
2      for (x = 0; x < 640; x++) {  
3          for (i = 0; i < 5; i++) {  
4              #pragma HLS PIPELINE II=1  
5              // do something 5 times ...  
6          }  
7      }  
    }
```

- ▶ Por defecto, los bucles en HLS quedan **enrollados** (rolled)... *todas las iteraciones reutilizan el mismo hardware.*
- ▶ **UNROLL** crea **copias** del cuerpo del bucle en RTL... *varias iteraciones ocurren en paralelo.*
- ▶ Mayor **throughput** y menor **latencia** por lote.
- ▶ **Mayor área/recursos** (LUTs, FFs, DSPs, BRAM).
- ▶ Si los límites del bucle son estáticos, puede hacerse **unroll total**; si no, usar **factor**.

Ejemplo de UNROLL

Sin optimización (rolled): una sola unidad hardware procesa todas las iteraciones.

```
1 LOOP_1: for (int i = 0; i < 10; i++) {  
2     y[i] = f(x[i]);  
3 }
```

Unroll total: paraleliza las 10 iteraciones (máximo rendimiento, máximo uso de recursos).

```
1 LOOP_1: for (int i = 0; i < 10; i++) {  
2     #pragma HLS UNROLL  
3     y[i] = f(x[i]);  
4 }
```

Unroll parcial (factor): equilibrio rendimiento/recursos.

```
1 LOOP_1: for (int i = 0; i < 10; i++) {  
2     #pragma HLS UNROLL factor=5  
3     y[i] = f(x[i]);  
4 }
```

```
1  #include "test.h"
2  dout_t test(din_t A[N]) {
3      dout_t out_accum=0;
4      dsel_t x;
5      LOOP_1:for (x=0; x<N; x++) {
6          out_accum += A[x];
7      }
8      return out_accum;
9  }
```

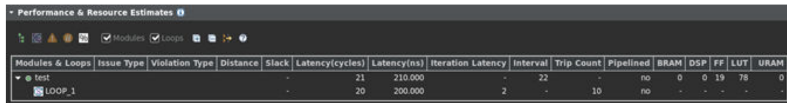
Performance & Resource Estimates

Modules ☒ Loops ☒

Modules & Loops	Issue Type	Violation Type	Distance	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
test			-		21	210.000	-	22	-	no	0	0	19	78	0
LOOP_1			-		20	200.000	2	-	10	no	-	-	-	-	-

Aplicando UNROLL

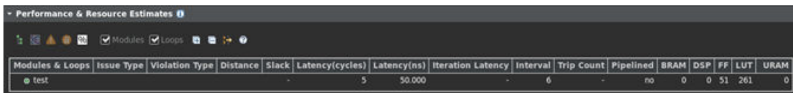
- Sin optimización: el *Synthesis Summary* muestra implementación secuencial; LOOP_1 con *trip count* = 10 y *latencia* = 200 ciclos (tiempo antes de aceptar nuevos datos de entrada).



Performance & Resource Estimates

Modules & Loops	Issue Type	Violation Type	Distance	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
test					21	210.000		22	-	no	0	0	19	78	0
LOOP_1					20	200.000		2	10	no	-	-	-	-	-

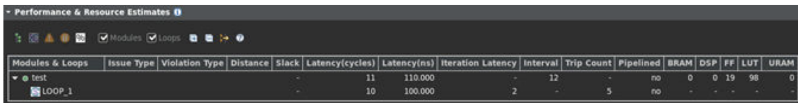
- Latencia lo más corta posible. *UNROLL totalmente* el bucle reduce drásticamente la latencia al ejecutar iteraciones en paralelo (50 ns), a cambio de mayor uso de recursos porque el bucle se reemplaza por copias hardware del cuerpo.
- Relación entre recursos vs. rendimiento.



Performance & Resource Estimates

Modules & Loops	Issue Type	Violation Type	Distance	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
test					5	50.000		6	-	no	0	0	51	261	0

- Si el *UNROLL* total no es viable por recursos, usa **UNROLL parcial** con factor *F*: paralelizas *F* iteraciones, sube el rendimiento y moderas LUT/FF/DSP.
- Usando **factor** = 2 duplica el cuerpo y reduce el *trip count* a la mitad (p.ej., 10 to 5).



Performance & Resource Estimates

Modules & Loops	Issue Type	Violation Type	Distance	Slack	Latency(cycles)	Latency(ns)	Iteration Latency	Interval	Trip Count	Pipelined	BRAM	DSP	FF	LUT	URAM
test					11	110.000		12	-	no	0	0	19	98	0
LOOP_1					10	100.000		2	5	no	-	-	-	-	-



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

HLD & HLS

Hardware Language Description & High Level Synthesis

Doctorado en Inteligencia Artificial

Procesadores de Alto Rendimiento para IA

Profesor: Jorge Portilla G.

Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile